SENSOR 接入指南

1.0.6

2023-08-04

上海富瀚微电子股份有限公司
地址：上海市徐汇区宜山路 717 号 2 号楼 5 楼-6 楼邮编：200233
网址：http://www.fullhan.com
传真：021-64066786
电话：021-61121558
邮箱：sales@fullhan.com

版权所有 © 上海富瀚微电子股份有限公司。保留一切权利。
非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标申明

Fullhan®、富瀚均为上海富瀚微电子股份有限公司的商标

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

目录

目录 1

1 适用范围 1-1

2 前言 2-1

2.1 修订记录 2-1

3 概述 3-1

4 Sensor 接口 4-1

4.1 变更 4-1

4.2 接口函数 4-1

4.2.1 函数说明 4-2

4.2.2 注意事项 4-11

4.2.3 接口实现说明 4-11

4.3 MIPI 接口说明 4-13

4.3.1 支持接入格式 4-15

4.3.2 接入条件 4-16

4.4 软件流程 4-16

4.5 代码示例 4-17

4.5.1 文件结构 4-17

4.5.2 创建 sensor 4-18

4.5.3 初始化流程 4-19

4.6 双路拼接 (FH8862 适用) 4-20

5 常见问题 Q&A 5-1

5.1 如何配置 sensor 幅面 5-1

5.1.1 参数解释 5-1

5.1.2 约束条件 5-1

5.1.3 示意图 5-2

5.2 启动时为何出现无法读写 sensor 的错误 5-2
5.3 如何检测配置的幅面是否正确 5-3
5.4 如何实时切换帧率 5-5
5.5 如何实时切换分辨率 5-6
5.6 实时切换制式时为何出现无 pic_end 中断错误 5-7
5.7 如何提高 sensor 启动速度 5-7
5.8 如何选择 sensor 输入时钟 5-7
5.9 如何抓取 sensor 的 raw 数据 5-7
5.10 如何导出 hex 文件 5-9
5.11 如何实现 get_sensor_attribute 接口 5-9

 版权所有 @ 上海富瀚微电子股份有限公司

1 适用范围

本文档适用于以下芯片：

- FH8862
- FH885xV300
- FH885xV310

版权所有 @ 上海富瀚微电子股份有限公司

2 前言

本文档主要描述了如何在 FULLHAN 平台下接入 sensor。

2.1 修订记录

修订时间	修订版本	修订人	修订描述
20221013	1.0.0	张晓飞	继承 FH889X V1.1.1
20221026	1.0.1	张晓飞	格式修订
20221117	1.0.2	张晓飞	更新 Sensor_Init_t 结构体
20221125	1.0.3	张晓飞	添加“如何实现 get_sensor_attribute 接口”章节
20221227	1.0.4	贺智佳	添加“接口实现说明”章节
20230801	1.0.5	张晓飞	添加支持的芯片型号
20230804	1.0.6	张晓飞	对不同芯片添加适用说明

版权所有 @ 上海富瀚微电子股份有限公司

3 概述

ISP 的固件主要包括系统控制单元，算法策略单元，sensor 单元及调试单元。系统控制单元主要用于处理中断，提供统计信息等；算法策略单元是 ISP 的核心，用于调整图像的整体效果，主要包括自动曝光，自动白平衡，色彩校正，去噪，锐化，坏点校正，宽动态，镜头阴影校正等；sensor 单元用于接入不同的 sensor；调试单元主要实现采用外部调试工具对寄存器进行读写的功能。

本文档主要描述了如何在 FULLHAN 平台下接入 sensor 的过程，在接入 sensor 的时候，需要按照下文描述的内容实现 FULLHAN sensor 相关的接口函数，最后运行相应的程序。在接入过程中遇到的一些常见问题，也在后面列了出来。

版权所有 @ 上海富瀚微电子股份有限公司

4 Sensor 接口

Sensor 需要向 ISP 库注册回调函数，用于实现与 ISP 的对接。每一个 sensor 都需要向 ISP 库注册回调函数，如 sensor 的初始化，曝光时间的控制，模拟增益的控制等等，ISP 通过这些回调函数实现对 sensor 的控制。

4.1 变更

相比之前的项目，现在每个 sensor 的接口多了一个 void 型的指针传入参数，传入的参数为 Sensor_Createxxx 创建的时返回的指针。该指针里面保留了创建 sensor 对象时候的多有信息，新增接口是为了一个 sensor 静态库的函数可以重用，从而 sensor 库内的变量是动态分配的，因此执行相应的函数的时候，需要传递对应的指针。

本项目支持双路拼接模式, 请参考后面章节的介绍.

4.2 接口函数

Sensor 回调函数的数据结构如下：

```
struct isp_sensor_if
{
    FH_CHAR*     name;
    FH_SINT32    (*get_vi_attr)(void *obj, ISP_VI_ATTR_S *vi_attr);
    FH_SINT32    (*set_flipmirror)(void *obj, FH_UINT32 sensor_en_stat);
    FH_SINT32    (*get_flipmirror)(void *obj, FH_UINT32 *sensor_en_stat);
    FH_SINT32    (*set_iris)(void *obj, FH_UINT32 iris);
    FH_SINT32    (*init)(void *obj, Sensor_Init_t* initConf);
    FH_VOID      (*reset)(void *obj, FH_VOID);
    FH_SINT32    (*deinit)(void *obj, FH_VOID);
    FH_SINT32    (*set_sns_fmt)(void *obj, int format);
    FH_SINT32    (*kick)(void *obj, FH_VOID);
    FH_SINT32    (*set_sns_reg)(void *obj, FH_UINT16 addr, FH_UINT16 data);
    FH_SINT32    (*set_exposure_ratio)(void *obj, FH_UINT32 exposure_ratio);
```

```
    FH_SINT32   (*get_exposure_ratio)(void *obj, FH_UINT32 *exposure_ratio);
    FH_SINT32   (*get_sensor_attribute)(void *obj, char * name,
    FH_UINT32 *value);
    FH_SINT32   (*get_sns_reg)(void *obj, FH_UINT16 addr, FH_UINT16 *data);
    FH_SINT32   (*get_awb_gain)(void *obj, FH_UINT32 *awb_gain);
    FH_SINT32   (*set_awb_gain)(void *obj, FH_UINT32 *awb_gain);
    SENSOR_USER_DEFINED_DATA*   data;
    FH_SINT32   (*common_if)(void *obj, unsigned int cmd,
    ISP_SENSOR_COMMON_CMD_DATA0* data0, ISP_SENSOR_COMMON_CMD_DATA1* data1);
    FH_SINT32   (*get_sns_ae_default)(void *obj,
    Sensor_AE_Default_S *sensAEDefault);
    FH_SINT32   (*get_sns_ae_info)(void *obj, Sensor_AE_INFO_S *sensAEInfo);
    FH_SINT32   (*set_sns_intt)(void *obj, FH_UINT32 intt, FH_UINT8 fNo);
    FH_SINT32   (*calc_sns_valid_intt)(void *obj, FH_UINT32 *u32Intt);
    FH_SINT32   (*set_sns_gain)(void *obj, FH_UINT32 again, FH_UINT8 fNo);
    FH_SINT32   (*calc_sns_valid_gain)(void *obj, FH_UINT32 *u32Gain);
    FH_SINT32   (*set_sns_frame_height)(void *obj, FH_UINT32 frameH);
    FH_UINT32*  (*get_sensor_mirror_flip_bayer_format)(void *obj, void);
    FH_UINT32*  (*get_user_awb_gain)(void *obj, FH_UINT32 idx);
    FH_UINT32*  (*get_user_ltm_curve)(void *obj, FH_UINT32 idx);
    FH_SINT32   (*is_sensor_connect)(void *obj);

};
```

4.2.1 函数说明

各函数的意义如下：

- name

  sensor 的名字，定义为一个字符串

- get_vi_attr(void *obj, ISP_VI_ATTR_S *vi_attr)

获取 sensor 的幅面等信息

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
ISP_VI_ATTR_S *vi_attr	结构体 ISP_VI_ATTR_S 为获取到的参数信息	OUT

```
typedef struct _ISP_VI_ATTR_
{
```

```
    FH_UINT16       u16WndHeight;    // sensor 幅面高度
    FH_UINT16       u16WndWidth;     // sensor 幅面宽度
    FH_UINT16       u16InputHeight;  // sensor 输入图像高度
    FH_UINT16       u16InputWidth;   // sensor 输入图像宽度
    FH_UINT16       u16OffsetX;      // 裁剪水平偏移
    FH_UINT16       u16OffsetY;      // 裁剪垂直偏移
    FH_UINT16       u16PicHeight;    // 处理的图像高度
    FH_UINT16       u16PicWidth;     // 处理的图像宽度
    FH_UINT16       u16FrameRate;    // 帧率
    ISP_BAYER_TYPE enBayerType;      // bayer 数据格式
} ISP_VI_ATTR_S;
```

- set_flipmirror(void *obj, FH_UINT32 sensor_en_stat)

配置 sensor 的水平翻转和镜像

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 sensor_en_stat	传入的水平和垂直镜像属性	IN

sensor_en_stat	水平翻转	镜像
0x1	是	否
0x2	否	是
0x3	是	是

- get_flipmirror(void *obj, FH_UINT32 *sensor_en_stat)

获取当前图像水平镜像及垂直翻转的状态，获取的值的对照值按上表。

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 *sensor_en_stat	获取到的水平和垂直镜像属性	OUT

- set_iris(void *obj, FH_UINT32 iris)

设置光圈大小，暂不支持

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 iris	光圈值	IN

- init(void *obj, Sensor_Init_t* initConf)

初始化 sensor，主要包括注册 sensor 设备号，重置 sensor，配置一些 sensor 的初始化寄存器。

版权所有 @ 上海富瀚微电子股份有限公司

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
Sensor_Init_t* initConf	传入的初始化参数	IN

```
typedef struct
{
    FH_UINT8 u8CsiDeviceId;  // 初始化的 MIPI 的设备号
    FH_UINT8 u8CciDeviceId;  // 当前和 sensor 实际链接的 I2C 设备号
    FH_BOOL  bGrpSync;       // 是否需要多 sensor 同步
} Sensor_Init_t;
```

- reset(void *obj)

复位 sensor

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN

- deinit(void *obj)

注销 sensor 的回调函数，各回调函数指针设为 NULL

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN

- set_sns_fmt(void *obj, int format)

  配置 sensor 的输出制式，参数为 format，是枚举变量，具体可以在 sensor 库中的 isp_enum.h 中可以看到，具体定义如下：

```
enum FORMAT
{
    FORMAT_720P25        = 0x03,
    FORMAT_720P30        = 0x04,
    FORMAT_960P25        = 0x0103,
    FORMAT_960P30        = 0x0104,
    FORMAT_1080P15       = 0x0201,
    FORMAT_1080P20       = 0x0202,
    FORMAT_1080P25       = 0x0203,
    FORMAT_1080P30       = 0x0204,
```


版权所有 @ 上海富瀚微电子股份有限公司

```
    FORMAT_1080P60       = 0x0207,
    FORMAT_1200P30       = 0x0304,
    FORMAT_1536X1536P15 = 0x0401,
    FORMAT_1536X1536P25 = 0x0403,
    FORMAT_1536X1536P30 = 0x0404,
    FORMAT_1536P15       = 0x0501,
    FORMAT_1536P25       = 0x0503,
    FORMAT_1536P30       = 0x0504,
    FORMAT_1536P40       = 0x0505,
    FORMAT_400WP15       = 0x0601,
    FORMAT_400WP20       = 0x0602,
    FORMAT_400WP25       = 0x0603,
    FORMAT_400WP30       = 0x0604,
    FORMAT_500W12P5      = 0x0700,
    FORMAT_500WP15       = 0x0701,
    FORMAT_500WP20       = 0x0702,
    FORMAT_800WP15       = 0x0801,
    FORMAT_270P240       = 0x0909,
    FORMAT_540P120       = 0x0a08,
    FORMAT_540P180       = 0x0a0c,
    FORMAT_2304X1296P25 = 0x0b03,
    FORMAT_192X192P220  = 0x0c0a,
    FORMAT_384X384P110  = 0x0d0b,
    FORMAT_WDR_1080P25  = 0x10203,
    FORMAT_WDR_1080P30  = 0x10204,
    FORMAT_WDR_1536P15  = 0x10501,
    FORMAT_WDR_1536P25  = 0x10503,
    FORMAT_WDR_1536P30  = 0x10504,
    FORMAT_WDR_1536P40  = 0x10505,
    FORMAT_WDR_400WP15  = 0x10601,
    FORMAT_WDR_400WP20  = 0x10602,
    FORMAT_WDR_400WP25  = 0x10603,
    FORMAT_WDR_400WP30  = 0x10604,
    FORMAT_WDR_500W12P5 = 0x10700,
    FORMAT_WDR_500WP15  = 0x10701,
    FORMAT_WDR_500WP20  = 0x10702,
    FORMAT_WDR_800WP15  = 0x10801,
    SENSOR_FORMAT_MAX   = 0xfffff,
};
```

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
int format	目标配置的制式	IN

- kick(void *obj)

部分 sensor 在下载完配置之后，需要配置 kick 寄存器才能够正常启动。

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN

- set_sns_reg(void *obj, FH_UINT16 addr, FH_UINT16 data)

往 senor 的指定地址，写入指定的数据。

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT16 addr	目标写的 sensor 寄存器地址	IN
FH_UINT16 data	目标写的 sensor 寄存器数据	IN

- (set_exposure_ratio)(void *obj, FH_UINT32 exposure_ratio)

  wdr 下配置长短帧曝光比例，wdr 策略中会传入该参数

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32 exposure_ratio	目标配置的曝光比例

- (get_exposure_ratio)(void *obj, FH_UINT32 *exposure_ratio)

  wdr 下获取长短帧曝光比例

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32* exposure_ratio	获取到的当前的曝光比例

- FH_SINT32 get_sensor_attribute(void *obj, char* name,FH_UINT32 *value)

  获取指定 name 的状态值

参数	释义
void *obj	Sensor_Createxxx 返回的指针
char* name	预期通过该接口获取的参数名
FH_UINT32* vale	获取到对应 name 的参数值
返回值	-1 获取失败，其他值获取正常

- FH_SINT32 (*get_sns_reg)(void *obj, FH_UINT16 addr, FH_UINT16 *data)

  获取 sensor 指定寄存器的值

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT16 addr	目标读的 sensor 寄存器地址

版权所有 @ 上海富瀚微电子股份有限公司

FH_UINT16* data	获取到指定地址的寄存器值

- FH_SINT32 (*get_awb_gain)(void *obj, FH_UINT32 *awb_gain)

  获取取到的是 sensor 当前配置的 awb 增益，仅支持 awb 增益的 sensor。

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32* awb_gain	获取 sensor 的当前 awb 增益

- FH_SINT32 (*set_awb_gain)(void *obj, FH_UINT32 *awb_gain);

  配置 sensor 的 awb 增益值，仅支持 awb 增益的 sensor.

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32* awb_gain	配置 sensor 的当前 awb 增益

- FH_SINT32 (*common_if)(void *obj, unsigned int cmd, ISP_SENSOR_COMMON_CMD_DATA0* data0, ISP_SENSOR_COMMON_CMD_DATA1* data1);

  自定义接口，通过 cmd 可以控制底层逻辑，支持传入的参数最大 128 个字的大小。

参数	释义
void *obj	Sensor_Createxxx 返回的指针
unsigned int cmd	定义的命令字
ISP_SENSOR_COMMON_CMD_DATA0* data0	入参结构体指针 0
ISP_SENSOR_COMMON_CMD_DATA1* data1	入参结构体指针 1

```
typedef struct _MAP_INTT_GAIN_CFG_S
{
    FH_SINT32 currFormat;  //当前的 sensor format | [0~0xffffffff]
    FH_SINT32 desFormat;   //目标的 sensor format | [0~0xffffffff]
    FH_UINT32 currIntt;    //当前的曝光时间，单位为行 | [0~0xffffffff]
    FH_UINT32 currTotalGain; //当前的曝光增益，u.6 精度 | [0~0xffffffff]
    FH_UINT32 desIntt;     //计算返回的曝光时间，单位为行 | [0~0xffffffff]
    FH_UINT32 desTotalGain; //计算返回的曝光增益，u.6 精度 | [0~0xffffffff]
} MAP_INTT_GAIN_CFG;

typedef struct _GPIO_PARAM_CFG_S
{
    FH_BOOL gpio_enable;  //第二阶段是否需要拉高拉低 GPIO | [0-0x1]
```


版权所有 @ 上海富瀚微电子股份有限公司

```
    FH_UINT32 gpio_time;  //第二阶段拉高拉低需要配置的延时    | [0-0xffffffff]
} GPIO_PARAM_CFG;

typedef struct _BLC_PARAM_CFG_S
{
    FH_UINT32 blc_value;  //配置的第一阶段 blc 值 | [0~0xffff]
} BLC_PARAM_CFG;

typedef union
{
    BLC_PARAM_CFG blc_param_cfg;
    GPIO_PARAM_CFG gpio_param_cfg;
    MAP_INTT_GAIN_CFG mapInttGainCfg;
    unsigned int dw[128];
} ISP_SENSOR_COMMON_CMD_DATA0;

typedef union
{
    unsigned int dw[128];
} ISP_SENSOR_COMMON_CMD_DATA1;
```

- FH_SINT32 (*get_sns_ae_default)(void *obj, Sensor_AE_Default_S *sensAEDefault)

  获取 AE 曝光 intt、again 最大最小值的限制，框架高度和最小复位行数。结构体 Sensor_AE_Default_S 的定义如下。

```
typedef struct _Sensor_AE_Default_S
{
    unsigned int  minIntt;  // 最小曝光时间，单位为行
    unsigned int  maxIntt;  // 最大曝光时间，单位为行
    unsigned int  minAGain;  // 最小曝光增益,U.6 精度
    unsigned int  maxAGain;  // 最大曝光增益,U.6 精度
    unsigned int  fullLineStd;  // 框架高度
    unsigned int  MinRstLine;  // 计算最大曝光时间时的最小复位行数
}Sensor_AE_Default_S;
```

- FH_SINT32 (*get_sns_ae_info)(void *obj, Sensor_AE_INFO_S *sensAEInfo)

  获取当前曝光状态参数，intt、again、行频、框架高度，结构体 Sensor_AE_INFO_S 如下。

版权所有 @ 上海富瀚微电子股份有限公司

```
typedef struct _Sensor_AE_INFO_S
{
    unsigned int  currIntt;  // 当前曝光时间，单位为行
    unsigned int  currAGain;  // 当前曝光增益，u.6 精度
    unsigned int  currHsf;  // 当前行频
    unsigned int  currFrameH;  // 当前框架高度
}Sensor_AE_INFO_S;
```

- FH_SINT32 (*set_sns_intt)(void *obj, FH_UINT32 intt, FH_UINT8 fNo)

  配置 senor 的曝光行数

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 intt	目标曝光行数	IN
FH_UINT8 fNo	在宽动态模式下使用，表示一帧当中不同的帧，长帧、中帧、短帧、超短帧等	IN

- FH_SINT32 (*calc_sns_valid_intt)(void *obj, FH_UINT32 * u32Intt)

  根据传入的曝光行数，返回实际 sensor 能够支持的曝光行数，对曝光行数有限制的 sensor 需要实现该函数。

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 * u32Intt	经过校正后的曝光值	IN/OUT

- FH_SINT32 (*set_sns_gain)(void *obj, FH_UINT32 again, FH_UINT8 fNo)

  配置 senor 的模拟增益

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 again	目标模拟增益	IN
FH_UINT8 fNo	在宽动态模式下使用，表示一帧当中不同的帧，长帧、中帧、短帧、超短帧等	IN

- FH_SINT32 (*calc_sns_valid_gain)(void *obj, FH_UINT32 *u32Gain)

  根据传入的模拟增益，返回实际 sensor 能够支持的模拟增益，对模拟增益有限制的 sensor 需要实现该函数。

版权所有 @ 上海富瀚微电子股份有限公司

参数	释义	参数传递方向
void *obj	Sensor_Createxxx 返回的指针	IN
FH_UINT32 *u32Gain	经过校正后的模拟增益	IN/OUT

- FH_SINT32 (*set_sns_frame_height)(void *obj, FH_UINT32 frameH)

  配置 sensor 的框架高度，支持慢快門的 sensor 需要实现该函数。frameH 的值，单位是行数，即目标配置的框架高度。

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32 frameH	目标配置的带消隐的总的高度

- FH_UINT32* (*get_sensor_mirror_flip_bayer_format)(void *obj)

  获取 sensor 在镜像、水平翻转之后的 bayer 的排列方式。

参数	释义
void *obj	Sensor_Createxxx 返回的指针
返回值	镜像或者水平翻转后对应的 bayer

- FH_UINT32* (*get_user_awb_gain)(void *obj, FH_UINT32 idx)

  获取用户自定义的 awb 增益。

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32 idx	序号为 idx 的 awb 增益

- FH_UINT32* (*get_user_ltm_curve)(void *obj, FH_UINT32 idx)

  获取用户自定义的 ltm 曲线。

参数	释义
void *obj	Sensor_Createxxx 返回的指针
FH_UINT32 idx	序号为 idx 的 ltm 曲线

- FH_SINT32 (*is_sensor_connect)(void *obj)

  RTT 下多 sensor 功能的时候，判断 sensor 是否连接。

参数	释义
void *obj	Sensor_Createxxx 返回的指针

版权所有 @ 上海富瀚微电子股份有限公司

4.2.2 注意事项

注意：

1. 初始化 sensor 时一定要注册 sensor 设备号，否则无法配置 sensor；
2. set_sns_fmt 一定要在初始化 sensor 之后，即 API_ISP_SetSensorFmt 要在 API_ISP_SensorInit 之后；
3. sensor 一些初始化寄存器的通用配置可以放入 init 中，也可以都放入 set_sns_fmt 中；
4. init 函数只能在上电的时候调用一次，若运行过程中重复调用，则会出现设备重复打开的错误；
5. 以上的各个回调函数可以根据需要选择实现，不一定每个 sensor 都需要实现所有的函数。

4.2.3 接口实现说明

以下接口必须实现：

```
// sensor 名字
FH_CHAR*    name;
// 获取 sensor 幅面信息
FH_SINT32   (*get_vi_attr)(void *obj, ISP_VI_ATTR_S *vi_attr);
// 初始化 sensor
FH_SINT32   (*init)(void *obj, Sensor_Init_t* initConf);
// 注销 sensor
FH_SINT32   (*deinit)(void *obj, FH_VOID);
// 配置 sensor 输出的制式
FH_SINT32   (*set_sns_fmt)(void *obj, int format);
// 获取指定状态值
FH_SINT32   (*get_sensor_attribute)(void *obj, char * name,
FH_UINT32 *value);
```

以下接口根据需求实现：

版权所有 @ 上海富瀚微电子股份有限公司

```
// 读写 sensor 寄存器相关回调函数
// 如不调用 API_ISP_SetSensorReg 和 API_ISP_GetSensorReg 等 api 可不实现
FH_SINT32   (*set_sns_reg)(void *obj, FH_UINT16 addr, FH_UINT16 data);
FH_SINT32   (*get_sns_reg)(void *obj, FH_UINT16 addr, FH_UINT16 *data);

// 曝光比例相关回调函数 wdr 策略中使用
// 非 wdr 出图可不实现
FH_SINT32   (*set_exposure_ratio)(void *obj, FH_UINT32 exposure_ratio);
FH_SINT32   (*get_exposure_ratio)(void *obj, FH_UINT32 *exposure_ratio);

// ae 相关回调函数
// 如不使用 fullhan ae，可不实现
FH_SINT32   (*get_sns_ae_default)(void *obj,
Sensor_AE_Default_S *sensAEDefault);
FH_SINT32   (*get_sns_ae_info)(void *obj, Sensor_AE_INFO_S *sensAEInfo);

// awb 增益相关回调函数
// 非 awb 增益的 sensor 可不实现
FH_SINT32   (*get_awb_gain)(void *obj, FH_UINT32 *awb_gain);
FH_SINT32   (*set_awb_gain)(void *obj, FH_UINT32 *awb_gain);

// 配置 sensor 曝光行数
// 如不调用 API_ISP_SetSensorIntt 且不使用 fullhan ae 可不实现
FH_SINT32   (*set_sns_intt)(void *obj, FH_UINT32 intt, FH_UINT8 fNo);

// 返回 sensor 实际曝光行数
// 对曝光行数有限制的 sensor 需要实现该函数
FH_SINT32   (*calc_sns_valid_intt)(void *obj, FH_UINT32 *u32Intt);

// 配置 sensor 模拟增益
// 如不调用 API_ISP_SetSensorGain 且不使用 fullhan ae 可不实现
FH_SINT32   (*set_sns_gain)(void *obj, FH_UINT32 again, FH_UINT8 fNo);

// 返回 sensor 实际模拟增益
// 对模拟增益限制的 sensor 需要实现该函数
FH_SINT32   (*calc_sns_valid_gain)(void *obj, FH_UINT32 *u32Gain);

// 配置 sensor 框架高度
// 支持慢快门的 sensor 需要实现
FH_SINT32   (*set_sns_frame_height)(void *obj, FH_UINT32 frameH);

// 配置和获取水平和垂直镜像，获取 sensor 在镜像后的 bayer 排列方式
// 有水平和垂直翻转需求时需要实现
FH_SINT32   (*set_flipmirror)(void *obj, FH_UINT32 sensor_en_stat);
FH_SINT32   (*get_flipmirror)(void *obj, FH_UINT32 *sensor_en_stat);
FH_UINT32*  (*get_sensor_mirror_flip_bayer_format)(void *obj, void);
```


版权所有 @ 上海富瀚微电子股份有限公司

```
// 复位 sensor
// sensor 有复位需求时需要实现
FH_VOID      (*reset)(void *obj, FH_VOID);

// 启动 sensor
// sensor 下载完配置需要配置启动寄存器时需要实现
FH_SINT32    (*kick)(void *obj, FH_VOID);

// 获取用户自定义的 awb 增益
FH_UINT32*   (*get_user_awb_gain)(void *obj, FH_UINT32 idx);

// 获取用户自定义的 ltm 曲线
FH_UINT32*   (*get_user_ltm_curve)(void *obj, FH_UINT32 idx);

// rtt 下多 sensor 判断 sensor 是否连接
FH_SINT32    (*is_sensor_connect)(void *obj);
```

4.3 MIPI 接口说明

在接入 mipi sensor 的过程中，需要在 Sensor_Init 函数中调用 mipi 相关的函数，对 sensor 进行初始化。

1. PHY 的初始化 (FH8862 适用)

本项目有两个 MIPI_PHY,MIPI_PHY0 固定为 1x4LANE 模式, 不可修改;MIPI_PHY1 支持初始化为 2x2LANE 模式，或者初始化为一个 1x4LANE 模式，驱动默认是把 MIPI PHY1 配置成 2x2LANE 模式的，如果把 MIPI_PHY1 配置为 1x4lane，则需要在应用程序启动前，运行下面的命令。

- 2x2LANE: 支持接入两个 2lane 的 sensor
- 1x4LANE: 支持接入 1 个 4lane 的 sensor

```
echo lane_4 > /proc/driver/mipi
```

2. 初始化的函数

   void mipi_init(struct mipi_conf *conf)

3. 结构体的说明

struct mipi_conf 的数据结构如下：

```
struct mipi_conf
{
    int mipi_id;     // MIPI 的设备号
    int frq_range;   // 当前 data lane 的速率
    int sensor_mode;// 当前 sensor 工作在宽动态还是线性模式
    int raw_type;    // 当前 MIPI 处理的数据格式，无效
    int lf_vc_id;    // 长帧的虚拟通道
    int sf_vc_id;    // 短帧的虚拟通道
    int lane_num;    // 当前 sensor 输出配置的 lane 数量
};
```

数据定义如下：

- mipi_id: MIPI 的设备号，不同的设备号对应不同的 MIPI 硬件
- frq_range：mipi 每条 lane 的时钟频率
- sensor_mode：用于选择 sensor 的类型，是否是 sony 的，是否是 wdr，sensor_mode 的枚举变量如下：

```
enum SENSOR_MODE
{
    NON_WDR = 0;
    NOT_SONY_WDR_USE_WDR = 1,
    SONY_WDR_NOT_USE_WDR = 2,
    SONY_WDR_USE_WDR = 3,
};
```

- raw_type：选择数据进来的位宽，10bit 还是 12bit，配置无实际影响，raw_type 的枚举变量如下：

版权所有 @ 上海富瀚微电子股份有限公司

```
enum RAW_TYPE
{
    RAW10 = 0,
    RAW12 = 1,
};
```

- lf_vc_id：长帧虚拟通道 id，每个 sensor 可能不一样，根据 sensor 文档定。
- sf_vc_id：短帧虚拟通道 id，也需参考 sensor 文档。
- lane_num: 初始化 mipi ctrl 的 lane 模式

1. 示例代码

下面以将 mipi0 初始化为宽动态的模式举例。

```
    struct mipi_conf config;
    config.mipi_id = 0;
    config.frq_range = R_800_849;
    config.sensor_mode = NOT_SONY_WDR_USE_WDR;
    config.raw_type = RAW10;
    config.lane_num = 4;

    config.lf_vc_id = 0;
    config.sf_vc_id = 1;

    mipi_init(&config);
```

4.3.1 支持接入格式

(FH8862 适用)

- 10bit raw，支持 online/offline，且 ISP 可以处理
- 12bit raw，支持 online/offline，且 ISP 可以处理
- 16bit raw, 仅仅支持 offline，ISP 不可处理
- 16bit yuv，仅仅支持 offline，ISP 不可处理

 版权所有 @ 上海富瀚微电子股份有限公司

(FH885xV300/FH885xV310 适用)

- 10bit raw
- 12bit raw

4.3.2 接入条件

1. 确保 SENSOR VS/HS/PIX_CLK 以及 D0~D15 信号输入正常
2. 确保以上管脚的 IO 复用正常，IO 驱动能力配置正常
3. 最后调用 VICAP 的接口配置 VI 属性正常接入 (FH8862 适用)

4.4 软件流程

Sensor 的启动流程如图 2-1 所示。主要分为以下几个步骤

1. 创建 sensor，根据需要实现 sensor 的接口函数。
2. 注册 sensor 至 ISP，调用 API_ISP_SensorRegCb 接口。
3. 初始化 sensor，调用 API_ISP_SensorInit 接口。
4. 配置 sensor 输出制式，调用 API_ISP_SetSensorFmt 接口。

版权所有 @ 上海富瀚微电子股份有限公司

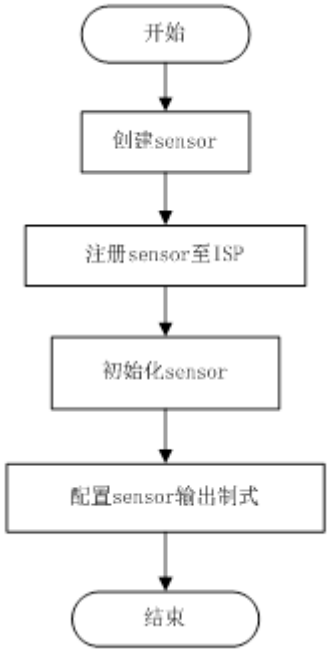

Figure 1: sensor 初始化流程

4.5 代码示例

4.5.1 文件结构

若要新接入一款 sensor，一般所要创建的文件有：

- xxx.h

  表示 sensor 相关的头文件，sensor 的名称以 xxx 表示，下同。

- xxx.c
- makefile
- hex 文件（保存白天及晚上的效果参数）

需要用到的库有：

- isp_sensor_if.h
- sensor_ops.h

 版权所有 @ 上海富瀚微电子股份有限公司

- isp_enum.h
- isp_func_def.h

4.5.2 创建 sensor

以 OVOS05 为例：

```
#define NAME(n) _NAME(n, ovos05_mipi)
#define SENSOR_OVOS05_MIPI    "ovos05_mipi"
#ifdef OS_RTT
struct isp_sensor_if ovos05_mipi_sensor_if __attribute__
((section(".sensor_drv"))) = {
#else
struct isp_sensor_if ovos05_mipi_sensor_if = {
#endif
    .name = SENSOR_OVOS05_MIPI,
    .get_vi_attr = GetSensorViAttr,
    .set_flipmirror = SetSensorFlipMirror,
    .get_flipmirror = GetSensorFlipMirror,
    .set_iris = SetSensorIris,
    .init = Sensor_Init,
    .reset = SensorReset,
    .deinit = Sensor_DeInit,
    .set_sns_fmt = SetSensorFmt,
    .kick = Sensor_Kick,
    .set_sns_reg = SetSensorReg,
    .set_exposure_ratio = SetExposureRatio,
    .get_exposure_ratio = GetExposureRatio,
    .get_sensor_attribute = GetSensorAttribute,
    .set_lane_num_max = SetLaneNumMax,
    .get_sns_reg = GetSensorReg,
    .get_awb_gain = GetSensorAwbGain,
    .set_awb_gain = SetSensorAwbGain,
    .data = 0,
    .common_if = SensorCommonIf,
    .get_sns_ae_default = GetAEDefault,
    .get_sns_ae_info = GetAEInfo,
    .set_sns_intt  = SetIntt,
    .calc_sns_valid_intt = CalcSnsValidIntt,
    .set_sns_gain = SetGain,
    .calc_sns_valid_gain = CalcSnsValidGain,
    .set_sns_frame_height = SetSnsFrameH,
    .get_sensor_mirror_flip_bayer_format = GetMirrorFlipBayerFormat,
    .get_user_awb_gain = GetUserSensorAwbGain,
```

 版权所有 @ 上海富瀚微电子股份有限公司

```
    .get_user_ltm_curve = GetSensorLtmCurve,
    .is_sensor_connect = Sensor_Isconnect,
};

#ifdef OS_RTT
#else
struct isp_sensor_if *NAME(Sensor_Create)()
{
    struct isp_sensor_if *s_if;
    struct sns_para_s    *para;

    s_if = (struct isp_sensor_if *)malloc(sizeof(struct isp_sensor_if));
    memcpy(s_if, &ovos05_mipi_sensor_if, sizeof(struct isp_sensor_if));

    para = (struct sns_para_s *)malloc(sizeof(struct sns_para_s));
    memcpy(&para->sns_cfg, &sns_cfg_default, sizeof(struct sns_cfg_s));
    memset(&para->sns_conf, 0, sizeof(SensorConfigPtr));
    memset(&para->sdev, 0, sizeof(SensorDevice));

    s_if->para = para;

    return s_if;
}

void NAME(Sensor_Destroy)(struct isp_sensor_if *s_if)
{
    free(s_if->para);
    s_if->para = 0;
    free(s_if);
    s_if = 0;
}
#endif
```

等号左边是 sensor 接口函数指针，等号右边是 ovos05 各个接口函数的具体实现。

4.5.3 初始化流程

sensor 的初始化流程示例代码为, 这个代码流程需要在

```
    FH_UINT32 u32IspDevId = 0;
    struct isp_sensor_if *sensor;
    int format = FORMAT_400WP25;
```

```
sensor = Sensor_Create_ovos05_mipi();

API_ISP_SensorRegCb(u32IspDevId, 0, sensor);

{
    Sensor_Init_t stSnsInit;
    stSnsInit.u8CsiDeviceId = 0;  // 初始化 MIPI0
    stSnsInit.u8CciDeviceId = 1;  // 选定 I2C1
    stSnsInit.bGrpSync = FH_FALSE;// 非拼接模式不需要同步
    API_ISP_SensorInit(u32IspDevId, &stSnsInit);
}

API_ISP_SetSensorFmt(u32IspDevId, format);
```

4.6 双路拼接 (FH8862 适用)

本项目支持双路拼接模式, 拼接模式下需要同步两路 sensor 的配置. 实现方法是, 修改之前的写 sensor 寄存器函数, 由直接写入硬件改为写入 buffer, 然后通过 common_if 接口将 buffer 中的数据发送给 ISP 驱动, 由 ISP 驱动实现同步配置的逻辑.

发送的数据结构定义如下:

```
//传递给 isp 的 sensor 寄存器配置
typedef struct {
    FH_UINT32        u32Id;         //内部使用, 不需要配置
    FH_BOOL          bNow;          //内部使用, 不需要配置
    FH_UINT8         u8Ccid;        //设备号
    FH_UINT32        u32DevAddr;    //I2C 驱动地址
    FH_UINT8         mode;          //I2C 数据模式
    FH_UINT32        u32RegNum;     //缓存数据个数
    SNS_REG_DATA_S   astRegData[SNS_REGS_DATA_MAX_NUM]; //缓存数据内容
    FH_BOOL          bValid;                            //内部使用, 不需要配置
} SNS_REGS_INFO_S;

//单个 sensor 寄存器配置
typedef struct
{
    FH_UINT32       u32Addr;     //寄存器地址
    FH_UINT32       u32Data;     //寄存器数据
    FH_BOOL         bNeedUpdate;//数据有效
```

```
    SNS_INT_POS_E   u8IntPos;   //配置时机点
} SNS_REG_DATA_S;

//配置时机点
typedef enum
{
    SNS_INT_POS_START    = 0, //一帧开始时
    SNS_INT_POS_END      = 1, //一帧结束时
}SNS_INT_POS_E
```


版权所有 @ 上海富瀚微电子股份有限公司

5 常见问题 Q&A

5.1 如何配置 sensor 幅面

对于 isp 中对应的 ISP_VI_ATTR_S 的配置，以 960p25 为例，如下：

```
u16WndWidth = 1600;
u16WndHeight = 1350;
u16InputWidth = 1280;
u16InputHeight = 960;
u16PicWidth = 1280;
u16PicHeight = 960;
u16OffsetX = 0;
u16OffsetY = 0;
```

5.1.1 参数解释

- u16WndWidth 图像框架幅面的宽度
- u16WndHeight 图像框架幅面的高度
- u16InputWidth 图像有效幅面的宽度
- u16InputHeight 图像有效幅面的高度
- u16PicWidth ISP 实际处理的图像宽度
- u16PicHeight ISP 实际处理的高度
- u16OffsetX 实际处理图像相对于图像有效幅面的水平方向的偏移位置
- u16OffsetY 实际处理图像相对于图像有效幅面的垂直方向的偏移位置

5.1.2 约束条件

- u16InputWidth >= u16PicWidth + u16OffsetX
- u16InputHeight >= u16PicHeight + u16OffsetY

 版权所有 @ 上海富瀚微电子股份有限公司

- u16WndWidth > u16PicWidth
- u16WndHeight > u16PicHeight

5.1.3 示意图

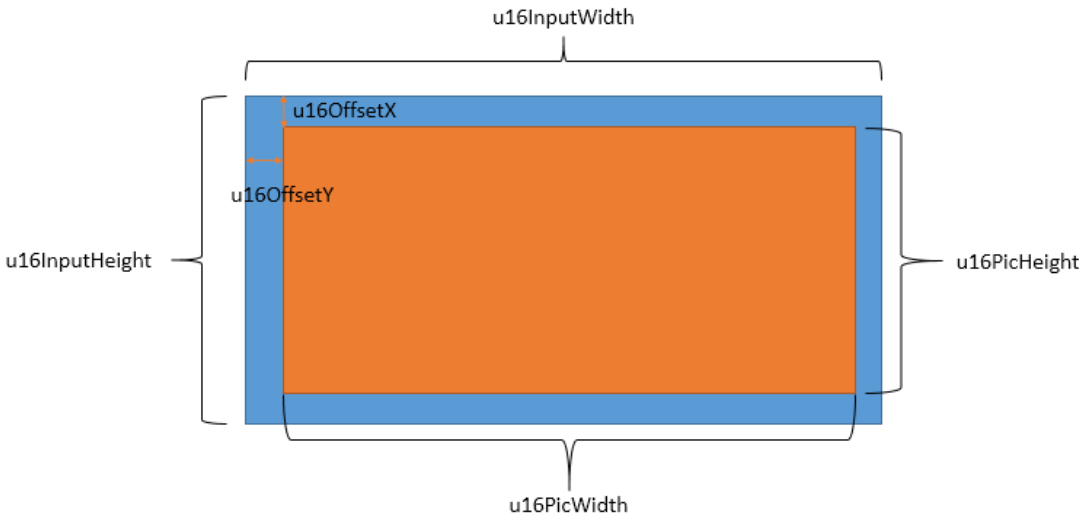

Figure 2: 各个参数关系示意图

5.2 启动时为何出现无法读写 sensor 的错误

客户在启动过程中出现

```
"ERROR: Unable to write sensor register!"
```

的打印信息。一般情况下，若遇到该问题，很有可能是 sensor 的设备号及模式初始化错误。

Sensor 设备初始化函数为 SensorDevice_Init(FH_UINT16 i2c_devid, FH_UINT8 mode)，第一个参数为设备号，由 sensor 厂商提供，写入时需右移一位，例如 IMX138，sony 提供的设备号为 0x34，则 i2c_devid = 0x1a；第二个参数为模式，对于 I2C 通信接口，范围为 0~3，定义如下：

Mode 0：8bit 地址，8bit 数据

Mode 1：8bit 地址，16bit 数据

Mode 2：16bit 地址，8bit 数据

Mode 3：16bit 地址，16bit 数据

5.3 如何检测配置的幅面是否正确

当配置完 sensor 运行时，若发现图像显示不正常，发生移位，花屏等现象，或者完全没有图像，此时应该先检查 sensor 输出的幅面是否与预期一致。硬件中有专门的寄存器存放了当前检测出的图像的框架幅面及有效幅面，用户在使用的时候可以使用 FULLHAN 提供的调试接口。

方法一：

通过命令行输入相应的指令，可以查看对应的调试信息。

linux 下，在运行完相应的 demo 程序之后，通过网络连接上设备后，按照下图命令进行操作，可以查看到相应的信息，该方法仅在在线模式下有效。

```
fh-linux# cat /proc/driver/isp

ISP Status:

        ------------------------ISP CommonInfo----------------------
        SDK VERSION :                   v0.0.0(g0000000)
        Driver Version:                 g6224ed4-dirty
        ISPCore Version:                g6224ed4-dirty
        ISP API Version:                g6224ed4-dirty
        Sensor  Version:
        ViInfo  Overflow:               0
        ------------------------------------------------------------
        Detect frame size:              8507x1250
        Detect input size:              1920x1080
                                        isp1
        ISP Resolution                  1920x1080
        Sensor FPS:                     10000/1599(6.25)
```

```
Offline Work status:            idle

------------------------ISP Count Info----------------------
ISPP PIC START Count:           246
ISPP PIC END Count:             245
ISPP Lf Start:                  0
ISPF PIC START Count:           246
ISPF PIC END Count:             245
ISPB PIC START Count:           245
ISPB PIC END Count:             244

------------------------ISP Error Info----------------------
ISPP OVERFLOW Count:            0
VPU BUSY Total Count:           0
AE STAT ERR Count:              0
ISPP FBUF CONFLICT Count:       0
ISPF FBUF CONFLICT Count:       0
ISPB DMU Err Count:             0
Vi Ecdc Err Count:              0

------------------------Reset Type Count--------------------
ISP Soft Reset Total Count:     0
Vi Overflow Count:              0
Pipeline not complished:        0
Proc Command Triggered:         0
Ioctl Triggered:                0
Interrupt Responce Delayed:     0
API Get Raw:                    0
Ec Dc Error:                    0
Ispf Buf Conflict Triggered:    0
Vpu Busy:                       0

------------------------ISP Process Time--------------------
ISPP Time Avg:                  4979847
ISPP Time Max:                  4980354
ISPP Time HistoryMax:           4980354
ISPB Time Avg:                  2849386
ISPB Time Max:                  2850344
ISPB Time HistoryMax:           2854953

------------------------HISTORY TOTAL COUNT-----------------
ISPP PIC_START Total Count:     246
ISPP PIC_END Total Count:       245
IPF PIC_START Total Count:      246
IPF PIC_END Total Count:        245
IPB PIC_START Total Count:      245
IPB PIC_END Total Count:        244
ISPP Lf Start Total Count:      0
```


版权所有 @ 上海富瀚微电子股份有限公司

```
------------------------Others------------------------------
ISP FUNC TRACE:                 OFF
AE LOG:                         OFF
AWB LOG:                        OFF
AUTO RESET:                     ON
WDR SEL:                        ON
WDR CIRBUFF :                   ON
WDR BUF NUM :                   1
WDR BUF DEPTH :                 572
NR3D SEL:                       ON
NR3D CIRBUFF :                  ON
```

方法二：调用 API_ISP_DetectPicSize 函数，若返回值为 0，则说明检测到的幅面与所配置的一致，若非 0，则不一致。可以简单的定位幅面与预期是否符合，详细的调试推荐使用上面提到的方法一。

5.4 如何实时切换帧率

以 IMX138 为例，假设当前输出为 960p25，需要切换至 960p30，可以通过改变图像的框架高度，从原来的 1350 变成 1125。需要注意的是，在改变 sensor 的框架高度前，需停止 ISP，待所有配置都完成后，再重新运行 ISP，虽然只是切换了帧率，输出的有效幅面不变，但在修改时还是需要先停止 VPU，完成后再重新使能。
参考代码如下：

```
FH_UINT32 u32IspDevId = 0;
FH_UINT32 u32GrpId = 0;

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Disable(u32GrpId));

API_ISP_Pause(u32IspDevId); //停止 ISP

switch(format)
{

    case FORMAT_960P25:
        format = FORMAT_960P30;
        break;
```

```
        case FORMAT_960P30:
            format = FORMAT_960P25;
        break;

    }

    API_ISP_SetSensorFmt(u32IspDevId, format);

    API_ISP_Resume(u32IspDevId); //运行 ISP

    ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Enable(u32GrpId, 0));
```

5.5 如何实时切换分辨率

以 IMX138 为例，假设当前输出为 960p25，需要切换至 720p25，两种分辨率的框架幅面是一致的，有效幅面需要改变，从 1280*960* 切换至 *1280* 720，ISP 切换的流程与上一小节一致，但 VPSS 处理的幅面需要相应修订，同样，需在 VPSS disable 的状态下进行配置，配置完成后再 enable。

VPSS 的参考代码如下：

```
FH_UINT32 u32GrpId = 0;
FH_VPU_SIZE vi_size;

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Disable(u32GrpId));

vi_size.vi_size.u32Width = 1280;

vi_size.vi_size.u32Height = 720;

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_SetViAttr(u32GrpId, &vi_size));

ERR_RETURN("FH_VPSS_Disable", FH_VPSS_Enable(u32GrpId, 0));
```


版权所有 @ 上海富瀚微电子股份有限公司

5.6 实时切换制式时为何出现无 pic_end 中断错误

若发现上电直接运行 960p 或者 720p 时，图像是正常的，但在运行过程中直接切换分辨率或者帧率的时候出现 “wait pic_end fail” 的打印信息，则有可能是时序上出了问题，建议可以在切换幅面时多加一些等待时间。

5.7 如何提高 sensor 启动速度

由于和 sensor 之间的通信采用 I2C 接口，I2C 的速率较慢，一般在 400kb/s，开机启动很大一部分时间是花在 sensor 寄存器的初始化上的，为了确保写入的正确性，每次写完一个 sensor 寄存器后，会等待一段时间，内核系统中自带的函数是 usleep，单位为 1μm，所以 usleep(1000) 即为 1ms，当发现启动速度非常慢时，可以适当减小其延时，一般可采用 usleep(300) 左右。

5.8 如何选择 sensor 输入时钟

调用 API_ISP_SetCisClk 接口可以实现 sensor 输入时钟的切换，需要注意的是必须在 API_ISP_Init 后使用，否则会出错。

5.9 如何抓取 sensor 的 raw 数据

当图像出现一些异常效果时，有时需要确认是 sensor 的输出源存在问题，还是后续的图像处理导致，这时就需要抓取 sensor 的 raw 数据。

API 接口中的 API_ISP_GetRaw 函数用于实现该功能。

函数原型为：

```
FH_SINT32 API_ISP_GetRaw(FH_UINT32 u32IspDevId, FH_BOOL strategy_en,
                         FH_VOID* pRawBuff, FH_UINT32 u32Size,
                         FH_UINT32 u32FrameCnt)
```


版权所有 @ 上海富瀚微电子股份有限公司

其中 strategy_en 用于控制策略是否关闭，pRawBuff 是用户分配的用于存放数据的内存，u32Size 是分配的内存大小，u32FrameCnt 是需要抓取的帧数。

示例代码如下：

```
FILE* rawfile;
unsigned char* rawbuff;
ISP_VI_ATTR_S vi_attr;
int raw_size;
FH_UINT32 u32Id;
char filename[]="raw_data.raw";

framecount=1;

rawfile = fopen(filename, "wb");

if (NULL == rawfile){
    printf("[ERROR]: cannot create file - %sn", filename);
    return -1;
}

API_ISP_GetViAttr(u32Id, &vi_attr);

raw_size = vi_attr.u16PicWidth*vi_attr.u16PicHeight*12/8;

rawbuff = (unsigned char*)malloc(raw_size * framecount);

if (NULL == rawbuff){
    printf("[ERROR]: not enough memory for dump raw data n");
    return -1;
}

API_ISP_GetRaw(u32Id, 0, rawbuff, raw_size * framecount, framecount );

fwrite(rawbuff, 1, raw_size * framecount, rawfile);

free(rawbuff);

fclose(rawfile);
```

注意：由于抓取一帧数据的时间大于一帧，所以当抓取的帧数大于 1 时，不能实现连续抓取，中间是跳帧的。

版权所有 @ 上海富瀚微电子股份有限公司

5.10 如何导出 hex 文件

当调试完成后，需要将 DRV 寄存器保存成 hex 文件，具体步骤如下：

1、coolview 选择工具-数据读写，参数配置界面如下：

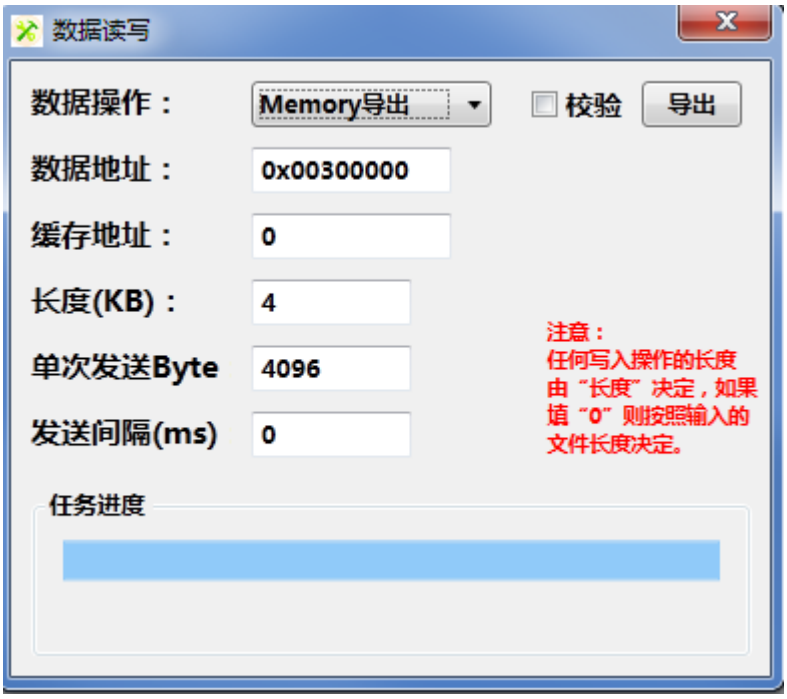

Figure 3: 导出参数界面示意图

2、保存之后，用 ultraEdit（或其他二进制文件读取软件）打开，将文件剪裁成 isp_param 结构体大小；

针对白天场景调试，导出白天的 hex 文件；然后再针对晚上切黑白的场景进行调试，导出晚上的 hex 文件。

5.11 如何实现 get_sensor_attribute 接口

通过接口 get_sensor_attribute 可获取 sensor 的各个属性配置:

```
"WDR" : 是否是 WDR 模式

"DCG" : 是否是 DCG 类型的 sensor
```

 版权所有 @ 上海富瀚微电子股份有限公司

示例如下:

```
static int GetSensorAttribute(void* pvObj, char * name, FH_UINT32 *value)
{
    FH_UINT32 u32WdrFlag = xx; //赋值 1 或 0, 表示是否是 WDR 模式;
    FH_UINT32 u32DcgFlag = xx; //赋值 1 或 0, 表示是否是 DCG 类型的 sensor;

    if (strcmp(name, "WDR") == 0)
        *value = u32WdrFlag;

    if (strcmp(name, "DCG") == 0)
        *value = u32DcgFlag;

    return 0;
}
```

 版权所有 @ 上海富瀚微电子股份有限公司

版权所有 @ 上海富瀚微电子股份有限公司